# Relation to Graph Concepts

Rajiv Sambasivan

September 11, 2025

## 1 Introduction

This document provides an overview of the concepts related to the transformation of relations into graph structures. We begin by defining the key terms and concepts that will be used throughout the discussion.

### 1.1 The Relational Model

[3] introduced the relational model, which is the logical foundation for relational databases. In this model, data is represented as relations (tables), where each relation consists of tuples (rows) and attributes (columns). An algebraic framework for manipulating these relations is provided by relational algebra, which includes binary operations (operate on two relations) such as join and union, as well as unary operations like selection and projection (operate on a single relation). The relational model provides a mathematical framework for representing and querying data.

### 1.2 The Entity-Relationship Model

The entity-relationship (ER) model, introduced by [1] is a framework for describing the *semantic* data abstractions and their relationship with each other. For example, if your use case is about how students in a university register for courses, then *students, course, registration* might be a set of data abstractions. The *registration* may connect a student with a course for a particular semester. From a *data representation* point of view, all semantic abstractions may be represented as relations.

### 1.3 The Entity-Relationship to Relational Schema Mapping

There exist guidelines and a body of knowledge around mapping an *Entity-Relationship Model* to a *Relational Schema*. See [5], for example. The working context for this document that you have a set of extracts from a relational database, in other words, this step has been done.

## 1.4 Use Case Data

Your *Use Case Data* is what you want to convert to a graph. The data for your use case is a set of extracts that have a semantic and database (data representational mapping). As a modeler, you are expected to know:

- The semantic abstractions relevant to your use case. These are the *entities* in your data extract. You should spend sometime to develop a clear understanding of the entities in your use case.
- The *semantic relationships* in your usecase. You should understand how *entities* in your usecase are related. You should spend sometime to understand the variations that are relevant from the perspective of your use case.
- The *semantic to extract mapping* for your usecase. You should know which extract file correspond to a particular entity or relationship. The extracts are what you get from the relational databases supplying the data for your use case. As an extension, we can look at connectors that fetch data from source relational systems, but for now we will assume you have file extracts of relations and entities.

## 1.5 Analysis

Given a use case, you first need to understand the characteristics of the data as it relates to your use case. In large datasets, there is often heterogeneity in the data. Heterogeneity means that the data is not uniform. For example, in a financial dataset, you may have data from different lines of business, different product categories, or different geographic regions. Each of these segments may have different characteristics that need to be accounted for in your analysis. Heterogeneity is discussed in detail in 2.2.

You may need several iterations of analysis to understand the data. You will need to evaluate different ways to assess the data from the perspective of your use case. Your use case may be associated with many metrics. For example, a revenue management use case may be associated with metrics such as revenue, profit, customer satisfaction, and market share. You will need to evaluate different ways to assess the data from the perspective of these metrics. A health care use case may be associated with metrics such as patient outcomes, cost of care, and patient satisfaction. You will need to evaluate different ways to assess the data from the perspective of these metrics.

For each perspective, you also need to evaluate different modeling approaches to see which one works best for your use case. Making an informed choice usually requires iterations to evaluate different modeling approaches from the perspective of desired performance metrics of your use case. Defining the performance metrics for your use case is an important step in this process.

The analysis step is the core focus of this document.

### 1.6 Modeling

Once you have a good understanding of the modeling approaches, heterogeneity in the data, and the performance metrics for your use case, you can start to build models. Evaluation of the modeling approaches is also part of the analysis step.

Once the approach is selected, you can start to build models. Operationalizing the model and monitoring the model is also part of the modeling step. This is typically done by an MLOps team. This is outside the scope of this document.

## 2 Guidelines for Implementation

The implementation consists of an initial assessment of the data, analysis for heterogeneity and a graph mapping. The guidelines for performing these steps are given below.

### 2.1 Initial Assessment

The initial assessment is done on the raw data obtained from the source systems. For the initial assessment, use a `jupyter notebook` to do the following:

- Inspection of Dataset: This is a quality assessment of the extracts you received for the entities and the relations. For each entity, verify that you have the required attributes in the extract. Make note of the desired attributes, you will need this to define the workflow to process the dataset.
- A *critical* point to note and understand at this point is the nature of analysis the use case is performing, i.e., are you doing a *Cross-Sectional, Longitudinal or Panel* study.
- Be clear about your perspective on heterogeneity in the data. You are typically in one of three situations:
    1. You have domain expertise about sources of heterogeneity in the data. You don't need to do any data-driven analysis to identify sources of heterogeneity.
    2. You don't have domain expertise about sources of heterogeneity in the data. You need to do data-driven analysis to identify sources of heterogeneity. You need to get this validated by domain experts.
    3. You believe that business conditions have changed and you need to identify new sources of heterogeneity in the data. You need to do data-driven analysis to identify sources of heterogeneity. You need to get this validated by domain experts.
- Understand and document the business rules that must be satisfied in your final data representation.For example, if you have a finish time and a start time, you may want to verify that the finish time is no earlier

than the start time. On financial data, signs, value-ranges etc., may need verification. Translate each rule to appropriate code. Sure there are data quality tools, but since there are only 5 entities per use case, the overhead of a data quality tool may not be worth it. Make note of how the business rules must be verified and how non-conforming records can be flagged, logged and dropped from the dataset.

- If you need numeric features for heterogeneity analysis, then you might want to featurize the dataset. If, for example you need to use clustering to group data into similar subsets, then you need to perform featurization where data for each entity chain is completely numeric. If you want to rely on segmentation, which translates to a group-by operation, to identify heterogeneity, you don't need to featurize at this point. So when you want to featurize a dataset will depend on how you want to do heterogeneity analysis.

The note comments in each of the steps above will be used to guide further processing.

## 2.2 Heterogeneity Analysis

The dataset that you created at the end of the previous step will be used for heterogeneity analysis. See [15] for details. The point about being clear about the type of analysis you want to perform for your use case will help with the analysis of heterogeneity. If you are using graphs for analysis, then heterogeneity is implicitly accounted for. However, it is still useful to understand the sources of heterogeneity in your data. This will help you understand the results of your analysis and also help you communicate the results to stakeholders. Therefore, it is useful to analyze the source dataset for heterogeneity.

In a *cross-sectional* analysis, sources of heterogeneity may be identified through domain expertise or by applying data-driven techniques. For instance, business datasets often exhibit heterogeneity across dimensions such as line of business, product category, or geographic region. To systematically uncover these sources, you can cluster the dataset based on variables that influence the outcome of interest. For example, if revenue is the key metric, performing feature selection—such as regressing revenue on various features—can help identify the most relevant variables. Clustering the data using these variables and profiling the resulting groups enables you to pinpoint and understand the main sources of heterogeneity.

In a *longitudinal analysis*, heterogeneity occurs over time. Tracking changes with respect to time using *change point analysis* can tell you when and how many changes have occurred in the period that you are monitoring. Of course, you will need collaboration with domain experts to validate the number and points of change. See [16] for an example. A reasonable approach to arrive at a set of candidate change points is to use an appropriate smoothing technique (a moving average filter, or another profiling method such as the *Matrix Profile*[18]), set up a decomposition[2] of the variable that is changing over time and then histogram the components of the decomposition. By counting the modes in each component of the decomposition and then adding them up, we get a baseline for the number of possible changes in the variable over time. Such a baseline can be discussed with domain experts and refined. The results from such a discussion can inform the *change point* detection algorithms.

In *Panel Data*, you need to account for sources of variation you have observed with both *cross-sectional* and *longitudinal* data.

Based on the principles discussed, partition the dataset from the previous step into meaningful subsets. The approach you use to create these partitions should align with the type of data analysis being performed—whether cross-sectional, longitudinal, or panel—and the specific objectives of your use case. For example, in supervised learning tasks with imbalanced data, you may need to apply specialized partitioning strategies compared to those used for balanced datasets. The primary goal is to simplify the analysis within each partition, which can be achieved by ensuring that each subset is as homogeneous as possible with respect to the outcome of interest. In supervised learning, this might involve creating partitions where simple models perform well; in unsupervised tasks such as dimensionality reduction, you might first cluster the data and then apply dimensionality reduction techniques within each cluster. Developing detailed guidelines for various scenarios is planned for future work. This step requires solid data analysis skills, and consulting with domain experts is highly recommended to ensure effective partitioning.

## 2.3 Graph Mapping

The challenge you are likely to face is the decision about how to map the source entities and relationships to a graph structure. There are three common analysis themes relating to graph structure that we frequently encounter in enterprise applications. The first theme corresponds to a *homogeneous graph*. In such a graph, the nodes are the same type of entity. In these graphs there is a *pivotal entity* that is the focus of the analysis. For example, in a financial use case, the pivotal entity may be a *customer*. In a health care use case, the pivotal entity may be a *patient*. In a manufacturing use case, the pivotal entity may be a *machine*. The analysis task is to understand the behavior of the pivotal entity. Understanding the behaviour of the pivotal entity is a major application of graph analysis in enterprise use cases. Please see [6] and [7] for a detailed discussion. When you want to connect two entities from the perspective of identifying heterogeneity in the relationship between the two entities (normal and anomalous behavior can be regarded as one example of this), then using a similarity measure or a distance metric to connect the two entities is a reasonable approach.

Another frequently encountered theme is the analysis of a *binary relationship*. For example, in a retail use case, we may be interested in understanding the relationship between *customer* and *product*. In a health care use case, we may be interested in understanding the relationship between *patient* and *treatment*. In a manufacturing use case, we may be interested in understanding the relationship between *machine* and *operator*. In such cases, we may want to use a graph structure that is centered around the two entities of interest. This is also a heterogeneous graph structure where the nodes are different types of entities. Specifically, this type of graph is a *bipartite* graph. Understanding how a pivotal entity interacts with another entity is a common analysis task in enterprise use cases. See [11] for a detailed discussion of theory and applications of bipartite graphs. Personalization is also common motivation for this type of analysis. See [12] for a detailed discussion of personalization.

Finally, the third common theme we counter is a small set of connected entities and relationships. The analysis tasks in this case are typically understanding the behavior of or trying to predict:

1. The property of an specific entity
2. The property of a relationship between two entities
3. The property of a collection of entities

For these problems, we can map the source entities and relationships to a graph structure that replicates the entity relationships in the source system. This is a heterogeneous graph structure where the nodes are different types of entities. The edges are the relationships between the entities. The tasks described above correspond to *node classification*, *link prediction* and *graph classification* tasks in the graph learning literature. See [19] for a detailed discussion. For a detailed discussion of graph representation learning, please see [9].

The analysis themes discussed above are by no means exhaustive. These are just the common themes that we have observed in enterprise use cases. Graphs have been used in many other ways to solve a variety of problems in engineering and science. For a discussion of other applications, for example to science and engineering, please see [4].

The analysis for heterogeneity is typically different for each of these three themes. Please see the examples section for some illustrative techniques. *Graphs do make it easier to account for heterogeneity in the analysis. This is primarily because most analysis methods consider a neighborhood of a node to predict a property or an edge connected to the node. The neighborhood of a node captures the heterogeneity information related to the node.* There is some upfront work to create the graph, but once the graph is created, the huge body of work on graph analysis can be brought to bear on the problem.

# 3 Knowledge Graph Construction

A review of the document so far will show that there are several steps in the production of a graph from relational data where modeler has to make decisions and choices. The modeler has to understand the use case, the data, the sources of heterogeneity in the data and the type of analysis that is required for the use case. The modeler also has to understand the different types of graph structures and their suitability for different types of analysis.

Knowledge graphs make it possible to capture the decisions and choices made by the modeler in a structured way. *Knowledge Management for Data Science (KMDS)* is a framework for capturing the decisions and choices made by the modeler in a structured way. The framework is implemented in Python and is available as an open source project on GitHub [14]. This framework needs work to integrate it with LLM tools.

However, the integration of KMDS with the steps outlined in this document is a work in progress. KMDS is mentioned here to highlight the importance of capturing the decisions and choices made by the modeler in a structured way.

# 4 Examples of Implementation

## 4.1 Homogeneous Graph from Loan Performance Data

The example is based the loan performance dataset for 7a loans from the Small Business Administration (SBA) (see this link). The dataset contains information about loans, borrowers, lenders, and loan performance. The *pivotal entity* in this dataset is the *borrower*. The analysis task is to understand the behavior of borrowers who default on their loans. The dataset is imbalanced, with a small number of borrowers who default on their loans. We want to understand borrowers who default on their loans.

## 4.2 Initial Assesment of the Data

1. See this notebook for an initial assessment of the data. This does the following:

- Checks data quality
- Removes data inconsistent with business rules
- For categorical attributes, using a simple heuristic of an outlier as something that has less than 5 occurrences, fixes outliers by either using the hierachy of the attribute (e.g., state) or replacing with "XXXX". For attributes with no hierarchy, they are dropped.
- Removes attributes that are not useful for analysis (e.g., IDs)
- ComputeTest train splits for evaluation purposes.
- Numerical encoding of categorical attributes (this notebook for details).

A note on the attributes with insufficient support is in order. The motivation for explicitly verifying that the attributes have sufficient support is that we want to account for generalization in the analysis. So a natural question to ask is what do we about the records that have attributes with low support? One strategy could be to build a model that excludes the attributes with low support. For example, if we have some states with a very small number of borrowers, we could build a model that excludes the geographic attributes and use that model to analyze the borrowers in those states that have few borrowers. When we have multiple attributes with low support, the number models that we have to build can grow quickly. In this analysis we have chosen to drop the records with attributes with low support. However, it is possible to work with domain experts and identify strategies to deal with attributes with low support. This is one more reason why automation of the analysis process is questionable at best. Corner cases abound and domain expertise is needed to deal with them. Another limitation of this analysis is that we have considered outliers in the attributes in a univariate manner. A multivariate analysis of outliers is possible, and ideally should be done. However, this is a more complex analysis and since this is an illustrative example, we have not done this analysis.

### 4.2.1 Heterogeneity Assemessment

Graphs permit us to capture heterogeneity implicitly, arguably automatically. The pivotal quantity here is the borrower. A homogeneous graph that captures all the attributes relevant to borrower is created.

In the training set, a $k$ neighborhood of the bad borrowers is created. This implicitly captures heterogeneity in the bad borrowers. The value of $k$ is determined by cross-validation. A value of $k = 2$ is used here, but since we have a validation dataset, this can be tuned. Since this is an example for illustration, hyper-parameter tuning is not done here. A rigorous analysis should include this. A bad borrower is characterized by the neighborhood of borrowers it is connected to.

Note that we have information about who the bad borrowers are. This is a supervised learning problem. The notion of a distance is used to compute the neighborhood. *Metric Learning*, see [10], for example for a detailed discussion, is used to learn a distance metric that separates the good borrowers from the bad borrowers. This computation factors the label information into the distance metric. *Neighborhood Components Analysis* [8] is used for metric learning. The distance metric is learned using the training dataset. The distance metric is then used to compute the k neighborhood of the bad borrowers.

The complement of the k neighborhood is the good borrower graph. See this notebook for the details of this analysis.

### 4.2.2 Graph Analysis

A basic classifier is created using the bad borrower graph. Let us pause and reflect on the structure of the bad borrower graph. The bad borrower graph is constructed by connecting the $k$ nearest neighbors of each bad borrower in the training dataset.

Since each bad borrower is associated with the same number of neighbors, the bad borrower graph is a regular graph. The degree of each node in the bad borrower graph is $k$. This maps to a tabular structure. Each row in the table is a bad borrower and his or her $k$ neighbors. A simple classifier is created using this tabular structure. The classifier is a random forest classifier. The random forest classifier is trained using the training dataset. The performance of the classifier is evaluated using the validation dataset.

As with any imbalanced dataset, you need to trade off precision and recall. In this situation, recall is more important than precision. A high recall means that we are able to identify most of the bad borrowers. The balanced accuracy is used as the performance metric. The balanced accuracy is the average of the recall and the specificity. The balanced accuracy is a good metric for imbalanced datasets. The balanced accuracy is 0.6 on the test dataset. The recall on the test dataset is 0.72. The precision on the test dataset is 0.14. The results are reasonable given the simplicity of the classifier. By incorporating costs, these results can be improved. The goal here is to illustrate the process of creating a graph from relational data and using the graph for analysis. Thresholding the probablity of a chargeoff based on the analysis of the ROC curve for the classifier and calibration of the classifier are some standard ways to improve performance of classifiers when working with imbalanced data, see [13]. These are not done here, since the goal is primarily to show that you can go grom a set of relations to a template that you can fine tune by following a recipe. See this notebook for the details of this analysis.

### 4.3 Bipartite Graph Construction from Relational Data

To do: An example of bipartite graph analysis from the Olist e-commerce dataset will be presented here.

### 4.4 Heterogeneous Graph Construction from Relational Data

To do: Please see [17] for an example of heterogeneous graph construction from relational data. The example uses service now data from a IT helpdesk. A heterogeneous graph is constructed from relational data. The graph is used to predict the resolution difficulty of an incident ticket. The paper is open access.

## References

[1] Peter Pin-Shan Chen. "The entity-relationship model—toward a unified view of data". In: *ACM transactions on database systems (TODS)* 1.1 (1976), pp. 9–36.

[2] William S Cleveland. "LOWESS: A program for smoothing scatterplots by robust locally weighted regression". In: *The American Statistician* 35.1 (1981), p. 54.

[3] Edgar F Codd. "A relational model of data for large shared data banks". In: *Communications of the ACM* 13.6 (1970), pp. 377–387.

[4] Diane J Cook and Lawrence B Holder. *Mining graph data*. John Wiley & Sons, 2006.

[5] Ramez Elmasri and Shamkant B Navathe. *Fundamentals of database systems seventh edition*. pearson, 2016.

[6] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=fMHZt4UStUg&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=1.

[7] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=eI-hmySej_w&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=2.

[8] J Goldberger et al. "Neighborhood components analysis, NIPS". In: *URL:< http://www. cs. utoronto. ca/ roweis/papers/ncanips. pdf* (2005).

[9] William L Hamilton. *Graph representation learning*. Morgan & Claypool Publishers, 2020.

[10] Brian Kulis et al. "Metric learning: A survey". In: *Foundations and Trends® in Machine Learning* 5.4 (2013), pp. 287–364.

[11] Jérôme Kunegis. "Exploiting the structure of bipartite graphs for algebraic and spectral graph theory applications". In: *Internet Mathematics* 11.3 (2015), pp. 201–321.

[12] Julian McAuley. *Personalized machine learning*. Cambridge University Press, 2022.

[13] probabl. *Cost Sensitive Learning*. Sept. 11, 2025. URL: https://probabl-ai.github.io/calibration-cost-sensitive-learning/content/notebooks/cost_sensitive_learning.html.

[14] Rajiv Sambasivan. *GitHub - rajivsam/KMDS: Source for KMDS — github.com*. https://github.com/rajivsam/kmds. [Accessed 06-09-2025].

[15] Rajiv Sambasivan. *Heterogeneity in Modeling*. Aug. 3, 2025. URL: https://rajivsam.github.io/r2ds-blog/posts/heterogeneity_in_models.

[16] Rajiv Sambasivan. *Understanding Coffee Prices*. https://rajivsam.github.io/r2ds-blog/posts/markov_analysis_coffee_prices/. Accessed: 2025-06-21. 2025.

[17] Jörg Schad, Rajiv Sambasivan, and Christopher Woodward. "Predicting help desk ticket reassignments with graph convolutional networks". In: *Machine Learning with Applications* 7 (2022), p. 100237. ISSN: 2666-8270. DOI: https://doi.org/10.1016/j.mlwa.2021.100237. URL: https://www.sciencedirect.com/science/article/pii/S2666827021001195.

[18] Chin-Chia Michael Yeh et al. "Matrix profile I: all pairs similarity joins for time series: a unifying view that includes motifs, discords and shapelets". In: *2016 IEEE 16th international conference on data mining (ICDM)*. Ieee. 2016, pp. 1317–1322.

[19] Jie Zhou et al. “Graph neural networks: A review of methods and applications”. In: *AI open* 1 (2020), pp. 57–81.